

RF-TRACE

Complete System Architecture&Implementation Phases

Smart India Hackathon 2026 | Problem Statement 26147

Automated model for analysis of .IQ and .wav files along with signal parameter extraction

1. Project Overview

RF-TRACE (Radio Frequency – Technical Recognition, Analysis&Characterization Engine) is an AI-assisted signal intelligence platform designed to analyze raw .IQ and .wav RF recordings and convert them into an interpretable signal profile.

Core principle: DSP measures. AI recognizes. Communication algorithms recover. Evidence fusion explains.

2. High-Level Architecture

The complete RF-TRACE pipeline is:

- 1. IQ/ WAV Input
- 2. File Ingestion
- 3. Signal Preprocessing
- 4. DSP Analysis + AI/CNN Recognition
- 5. Evidence Fusion
- 6. Signal Parameter Extraction
- 7. Demodulation
- 8. Bit Stream Recovery
- 9. De-interleaving Analysis
- 10. FEC Analysis + Viterbi
- 11. Bit Correlation&Structure Analysis
- 12. Signal Profile + Final Report

3. Architecture Layers

Layer	Responsibility
Input&Acquisition	IQ/WAV files, metadata, file validation, signal-format detection.
Signal Processing	Preprocessing, normalization, filtering, FFT, PSD, STFT, waterfall, constellation, segmentation.
AI Recognition	Automatic modulation classification using a PyTorch CNN and confidence/probability output.

Evidence Fusion	Combines CNN, spectral, constellation, time-domain and frequency-domain evidence.
Digital Signal Recovery	Synchronization, demodulation, symbol extraction and raw-bit recovery.
Communication Analysis	Interleaving/de-interleaving analysis, FEC analysis and Viterbi decoding.
Signal Intelligence	Bit correlation, pattern/structure analysis, signal profile and reporting.

4. Complete 12-Phase Architecture

Phase 1 — File Ingestion

Accept .IQ and .wav files; detect file type and IQ/audio layout; extract available metadata; convert input to a common complex-IQ representation where appropriate. Output: Signal Buffer containing samples, sample rate when available, center frequency when available, duration, format, channels and metadata.

- File type detection
- Format detection
- Metadata extraction
- Sample conversion
- Validation

Phase 2 — Signal Preprocessing

Prepare raw data for analysis through DC-offset removal, normalization, optional filtering, noise estimation, signal-activity detection and segmentation.

- DC offset removal
- Amplitude normalization
- Optional low/high/band-pass filtering
- Noise estimation
- Signal activity detection
- Segmentation

Phase 3 — DSP Analysis Engine

Measure physical and signal-processing characteristics. Generate waveform, FFT/PSD, waterfall/STFT and constellation views; estimate bandwidth, noise floor, power, phase/frequency characteristics and other features.

- Time-domain analysis
- FFT/PSD
- Waterfall/STFT
- Constellation

- Bandwidth and spectral features
- Signal statistics

Phase 4 — AI/CNN Modulation Recognition

Classify supported modulation types from IQ data. Initial MVP classes: BPSK, QPSK, 8PSK, FSK and 16QAM. Train a PyTorch CNN using labelled synthetic IQ data with realistic impairments.

- Dataset generation
- Train/validation/test split
- CNN training
- Evaluation
- Confusion matrix
- Model export as modulation_model.pt
- Inference

Phase 5 — Evidence Fusion

Combine independent observations instead of relying solely on the CNN. Produce a modulation hypothesis with supporting evidence and confidence.

- CNN probability
- Constellation evidence
- Spectral evidence
- Phase/time-domain evidence
- Consistency checks
- Hypothesis + confidence

Phase 6 — Signal Parameter Extraction

Create a structured parameter profile. Every result should identify provenance: measured, estimated, metadata-derived, AI-inferred, candidate/hypothesis or unknown.

- Sampling frequency
- Center frequency where available
- Bandwidth
- Modulation
- Symbol rate
- Carrier offset
- Frequency deviation
- SNR
- Duration
- Occupied bandwidth
- FEC/interleaving candidates

Phase 7 — Demodulation

Convert an identified digital modulation into symbols and raw bits through synchronization, frequency/carrier correction, timing recovery, symbol extraction and modulation-specific demodulation.

- BPSK
- QPSK
- 8PSK
- FSK
- 16QAM
- Symbol recovery
- Raw-bit generation

Phase 8 — Bit Stream Recovery

Present recovered bits in binary, hexadecimal and optional ASCII views, together with bit-level statistics and pattern information.

- Binary stream
- Hex view
- Optional ASCII
- Bit count
- 0/1 ratio
- Transitions
- Entropy estimate
- Repeated sequences

Phase 9 — De-interleaving Analysis

Test supported interleaving hypotheses and reverse bit ordering where evidence supports it. Initial candidates: block, convolutional, diagonal and pseudo-random.

- Candidate generation
- Candidate de-interleaving
- Result evaluation
- Candidate ranking
- Confidence/evidence

Phase 10 — FEC Analysis

Analyze supported Forward Error Correction schemes. MVP priority is convolutional coding with Viterbi decoding; Reed-Solomon, concatenated codes and LDPC remain modular extensions.

- Convolutional code
- Viterbi decoder
- Reed-Solomon extension
- Concatenated-code extension
- LDPC extension
- Candidate assessment

Phase 11 — Bit Correlation & Structure Analysis

Investigate the recovered bit stream for repeated or periodic structures without assuming a known protocol.

- Autocorrelation

- Repeated-pattern detection
- Preamble candidates
- Header/payload hypotheses
- Bit statistics
- Structure candidates

Phase 12 — Signal Profile & Final Report

Combine all results into an interpretable signal-intelligence report containing parameters, visualizations, modulation assessment, demodulated bits, FEC/interleaving findings, correlation results and evidence.

- Signal summary
- Waveform
- Spectrum
- Waterfall
- Constellation
- Modulation confidence
- Bit stream
- FEC/interleaving findings
- Correlation findings
- Exportable report

5. CNN Training Architecture

Signal Generator → BPSK / QPSK / 8PSK / FSK / 16QAM

→ Add realistic impairments: noise, SNR variation, frequency offset, phase offset, amplitude variation and timing variation

→ Labelled IQ dataset

→ 70% training / 15% validation / 15% test (initial target)

→ PyTorch CNN

→ Evaluation: accuracy, precision, recall, F1 and confusion matrix

→ Save models/modulation_model.pt

→ Connect model to RF-TRACE inference

6. Backend Module Architecture

RF-TRACE/

|—— frontend/

| |—— React

| |—— TypeScript

| |—— Vite

| |—— Tailwind

|—— backend/

```
| |───main.py
| |───api/
| |   ├──upload.py
| |   ├──analysis.py
| |   └──results.py
| |───signal/
| |   ├──loader.py
| |   ├──preprocessing.py
| |   ├──spectrum.py
| |   ├──waterfall.py
| |   ├──constellation.py
| |   └──parameters.py
| |───ai/
| |   ├──model.py
| |   ├──classifier.py
| |   ├──inference.py
| |   └──training/
| |       ├──generate_dataset.py
| |       ├──dataset.py
| |       ├──train.py
| |       ├──evaluate.py
| |       └──confusion_matrix.py
| |───fusion/
| |   └──evidence_fusion.py
| |───demodulation/
| |   ├──bpsk.py
| |   ├──qpsk.py
| |   ├──psk.py
| |   ├──fsk.py
| |   └──qam.py
| |───interleaving/
| |   ├──block.py
| |   ├──convolutional.py
| |   ├──diagonal.py
| |   └──pseudo_random.py
| |───fec/
| |   ├──convolutional.py
| |   ├──viterbi.py
| |   ├──reed_solomon.py
| |   └──ldpc.py
| |───correlation/
| |   └──bit_analysis.py
| |───reports/
| |   └──signal_profile.py
|───models/
|   └──modulation_model.pt
|───tests/
|───sample_data/
|───README.md
```

7. Frontend Architecture

Recommended navigation:

- Dashboard
- Signal Analysis
- Spectrum
- Waterfall
- Constellation
- Modulation
- Bit Stream
- FEC / Interleaving
- Signal Profile
- Settings

The UI should behave as a technical RF workstation rather than a generic SaaS dashboard.

8. API Architecture

Endpoint	Purpose
POST /api/upload	Upload .IQ/.wav file
POST /api/analysis	Start analysis
GET /api/analysis/{id}/status	Return processing progress
GET /api/analysis/{id}/results	Return complete signal profile

9. Controlled End-to-End Demonstration

For the hackathon, use a known ground-truth signal to demonstrate the complete chain:

13. Original bits
14. Convolutional FEC encoding
15. Interleaving
16. QPSK modulation
17. Noise/ frequency/ phase impairment
18. IQ recording
19. RF-TRACE ingestion and preprocessing
20. DSP+ CNN recognition
21. Evidence fusion
22. Demodulation
23. De-interleaving
24. Viterbi decoding
25. Recovered bits

- 26. Comparison against original bits
- 27. Final Signal Profile

10.24-Hour Hackathon MVP Priorities

Must Work

- IQ/WAV upload
- File parsing
- Preprocessing
- FFT/Spectrum
- Waterfall
- Constellation
- Parameter extraction where inferable
- CNN modulation classification
- Confidence display
- Several demodulators
- Bit-stream visualization
- Controlled FEC demonstration
- Controlled de-interleaving demonstration
- Final Signal Profile

Should Work

- Symbol-rate estimation
- SNR estimation
- Carrier-offset estimation
- Pattern detection
- Preamble candidate detection
- Report export

Do Not Attempt to Perfect in 24 Hours

- Universal blind FEC identification
- Every possible interleaver
- Every modulation standard
- Every communication protocol
- Perfect blind sampling-rate recovery
- Universal protocol decoding
- Real-time SDR deployment

11. Testing Strategy

Test	Ground-truth flow
------	-------------------

BPSK	Known bits → BPSK → IQ → RF-TRACE → BPSK detection
QPSK	Known bits → QPSK → IQ → RF-TRACE → QPSK detection → demodulation → bits
FSK	Known bits → FSK → IQ → RF-TRACE → FSK detection
16QAM	Known bits → 16QAM → IQ → RF-TRACE → 16QAM detection
FEC	Known bits → convolutional encoder → QPSK → noise → RF-TRACE → Viterbi → recovered bits
Interleaving	Known bits → block interleaver → modulation → RF-TRACE → demodulation → de-interleaving → original ordering

12. Data Provenance and Reliability Rules

- Never fabricate a signal parameter.
- Mark each parameter as measured, estimated, metadata-derived, AI-inferred, candidate/hypothesis or unknown.
- Do not claim universal modulation recognition when the CNN is trained only on supported classes.
- Treat blind FEC and interleaver identification as candidate/hypothesis analysis unless verified.
- Preserve individual evidence sources behind every fused assessment.
- Gracefully report insufficient signal quality or unsupported formats.

13. Core Innovation

The key differentiator is Multi-Evidence Signal Intelligence: RF-TRACE combines signal-processing measurements, learned recognition, communication algorithms and evidence-based confidence rather than depending solely on an AI classifier.

Core model: Hypothesis → Evidence → Confidence

14. Final Architecture Summary

File Ingestion → Preprocessing → DSP Analysis + CNN → Evidence Fusion → Parameter Extraction → Demodulation → Bit Recovery → De-interleaving → FEC → Bit Correlation → Signal Profile.

RF-TRACE — From Raw Signal to Signal Intelligence.

15. Implementation Sequence

Stage	Implementation scope
-------	----------------------

Foundation	Phases 1– 2: ingestion and preprocessing
Signal Visualization	Phase 3: waveform, FFT, spectrum, waterfall, constellation
AI	Phases 4– 5: CNN training/inference and evidence fusion
Parameter Intelligence	Phase 6: structured parameter extraction and provenance
Digital Recovery	Phases 7– 8: demodulation and bit recovery
Communication Analysis	Phases 9– 10: de-interleaving and FEC/Viterbi
Signal Intelligence	Phases 11– 12: correlation, structure analysis and final report